Universidade do Minho

Ano Letivo: 2025/26

Mineração de Dados

Ontologies and Knowledge Graphs

Miguel Rocha | mrocha@di.uminho.pt
Diana Ferreira | diana@di.uminho.pt

Contents

1 Foundations and Motivation

2 Ontologies

3 Knowledge Graphs

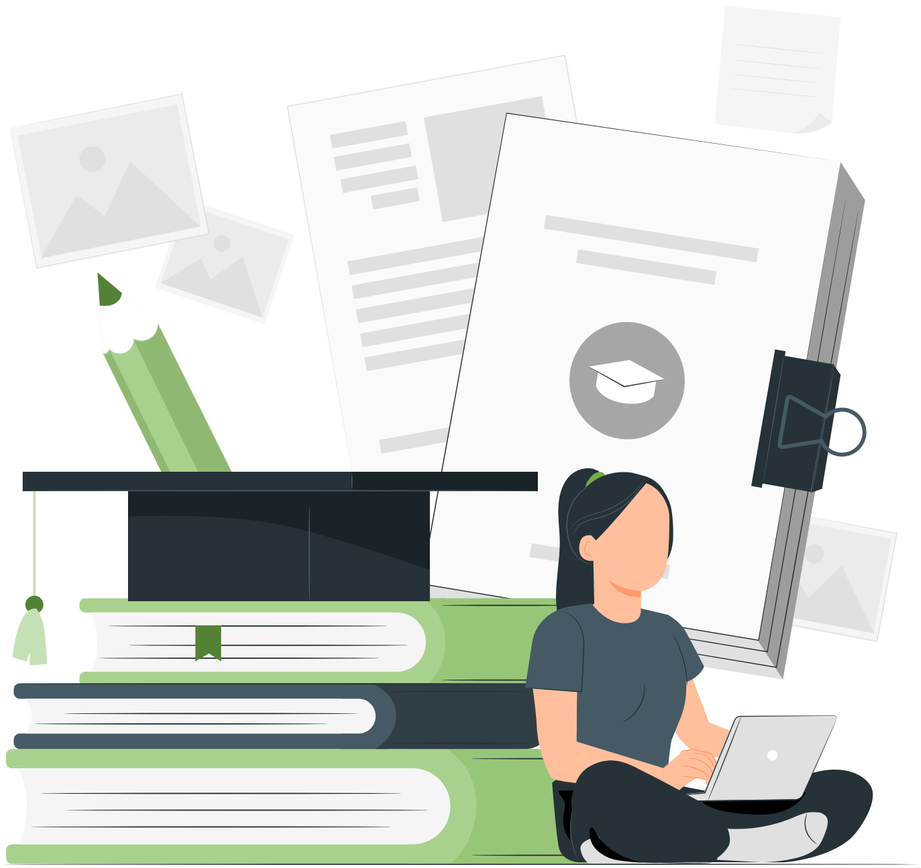
4 Applications

Learning Objectives

By the end of this lecture, students should be able to:

- Understand Ontologies and Knowledge Graphs
- Distinguish the two Knowledge Graph Models
- Understand RDF and RDFS, as well as SPARQL
- Understand neo4j and Cypher
- Comprehend the different applications of Knowledge Graphs in the real-world

Bibliography



Books:

1. Robinson, I., Webber, J., & Eifrem, E. (2015). *Graph databases: new opportunities for connected data.* " O'Reilly Media, Inc.".
2. Jordan, G. (2014). *Practical Neo4j*. Apress.
3. Tiwari, S., Rodriguez, F. O., Abbès, S. B., Usip, P. U., & Hantach, R. (Eds.). (2023). *Semantic AI in knowledge graphs*. CRC Press.

Official Documentation:

1. Neo4j / Cypher: https://neo4j.com/docs/cypher-manual/current/
2. RDF/SPARQL:
   - https://www.w3.org/TR/2025/WD-sparql12-protocol-20250814/
   - https://github.com/w3c/rdf-interop

Foundations and Motivation

What is knowledge?

WISDOM
Why? Judgment, insight

KNOWLEDGE
How? Patterns, models

INFORMATION
Who/What? Structured

DATA
Raw symbols, facts

Key Question:

How do machines represent and reason with knowledge?

Foundations and Motivation

➡ The Semantic Gap: From Syntax to Meaning

SYNTACTIC DATA – What Machines See

Structure over meaning: Machines are excellent at processing structure, format, and rules.

Examples: Tables, JSON/XML, Strings

The Limitation: A machine can validate that "Apple" is a string, but it doesn't "know" if it's a fruit or a company.

SEMANTIC DATA – What Users Want

Meaning & Context: The intended meaning behind the data.

Human Understanding:

- *"Apple"* → Context determines if we mean a tech giant or a piece of fruit.

The Problem: Machines Process Symbols, Not Meaning

Introduction of "Semantics": The term was adopted to describe the study of the relationship between symbols (syntax) and their interpretations (meaning). It aims to move data from purely structural representation to *actionable knowledge*.

Foundations and Motivation

A brief history

1960s–1980s

Symbolic AI

Emulate human decision-making using logic and manually coded rules (IF-THEN statements).

1980s

Description Logics (DLs)

A formal language for representing knowledge, defining concepts, roles, and individuals logically (e.g., "A Mother is a Woman who has a Child").

1990s

The Ontology Boom in AI

Moving from domain-specific logic to reusable, shared vocabularies for machines.

2001

The Semantic Web Vision (Tim Berners-Lee)

Extend the web of documents (HTML) into a web of linked data. Standards like **RDF** and **OWL** give data explicit meaning.

2012

Knowledge Graphs

Google launches the **Knowledge Graph**
Focus shifted to integrating massive, heterogeneous data sources.

2020s

The LLM + KG Hybrid Era

Mitigate LLM hallucinations and enhance reasoning capabilities by grounding them in structured semantic knowledge.

Trend: Moving from *hand-coded rules → formal standards → web-scale graphs → neuro-symbolic AI.*

Foundations and Motivation

➡ Motivation

❌ **The Problem with Isolated Data:** Traditional databases store data in silos, structured but disconnected.

↳ Real-world knowledge is inherently **relational**.

MOTIVATIONS:

- **Semantic richness** — Capture meaning and context, not just raw data
- **Relationships matter** — Model connections between entities the way humans naturally think
- **Heterogeneous data integration** — Unify data from diverse sources under a common framework
- **Inference & reasoning** — Derive new facts from existing ones using logic and ontologies
- **Flexibility** — Schema-free or schema-light design adapts to evolving domains
- **Explainability** — Transparent, traceable paths between connected facts

💡 **Core Insight:** Data becomes *knowledge* when enriched with structure, semantics, and relationships.

Ontologies

➡ What is an Ontology?

From the Greek **ontos** ("being") and **logia** ("study").

Question: What exists? How can we categorize the world?

Traditional Definition: The philosophical study of the nature of being, existence, and reality.

Computer Science Definition (Gruber, 1993): *"An ontology is a **formal**, **explicit** specification of a **shared conceptualization**."*

- ✅ **Conceptualization:** An abstract model of some phenomenon in the world.
- ✅ **Explicit:** All concepts and constraints are clearly defined.
- ✅ **Formal:** Machine-readable (not natural language).
- ✅ **Shared:** Captures consensual knowledge accepted by a community.

Ontologies

➡ What is an Ontology?

➔ **Analogy:** A Shared Dictionary + Grammar

Dictionary: Provides the vocabulary (concepts and terms) for a domain.

Example: Defines what "Doctor," "Patient," and "Hospital" mean.

Grammar: Provides the rules for how terms can relate.

Example: A "Doctor" *treats* a "Patient." A "Patient" *is admitted to* a "Hospital."

💡 **Together, they eliminate ambiguity and enable machines to interpret data consistently.**

Ontology = a theory of the kinds of things there are and can be (WHAT CAN EXIST)

Ontologies

Why Ontologies Matter?

Shared Understanding

Aligns terminology across teams and systems.

Everyone agrees on what "Active Patient" means.

Interoperability

Enables data exchange between heterogeneous systems.

System A's "M" = System B's "male."

Reusability

Publish once, use across multiple applications.

Don't reinvent the wheel; import standard medical ontologies.

Automated Reasoning

Infer new insights from existing data.

If X lives in Paris and Paris is in France, then X lives in France.

Use case: *Two hospital systems sharing patient records — ontology enables semantic mapping.*

Ontologies

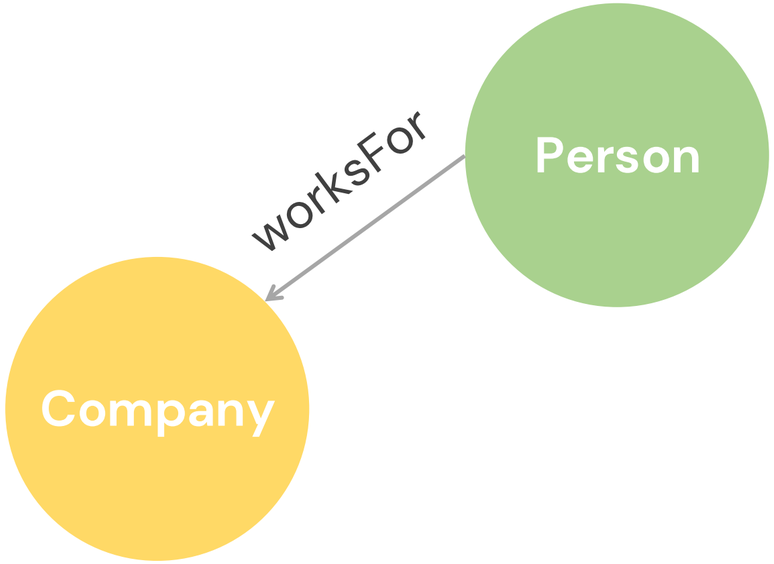
Anatomy of an Ontology

1. Classes (Concepts / Categories)
Represent what *types* of things exist in the domain.

- *Examples:* Person, Organization, City
- *Visual:* **Ovals/Rectangles** (e.g., [Person])

2. Properties (Relationships & Attributes)
Connect classes or link data to individuals.

- Object Properties: Link an individual to another individual.
  Example: hasChild (Person → Person),
  worksFor (Person → Organization)
- Datatype Properties: Link an individual to a literal value (string, number, date).
  Example: hasAge (Person → Integer),
  hasName (Person → String)
- *Visual:* **Arrows** labeled with the property name.



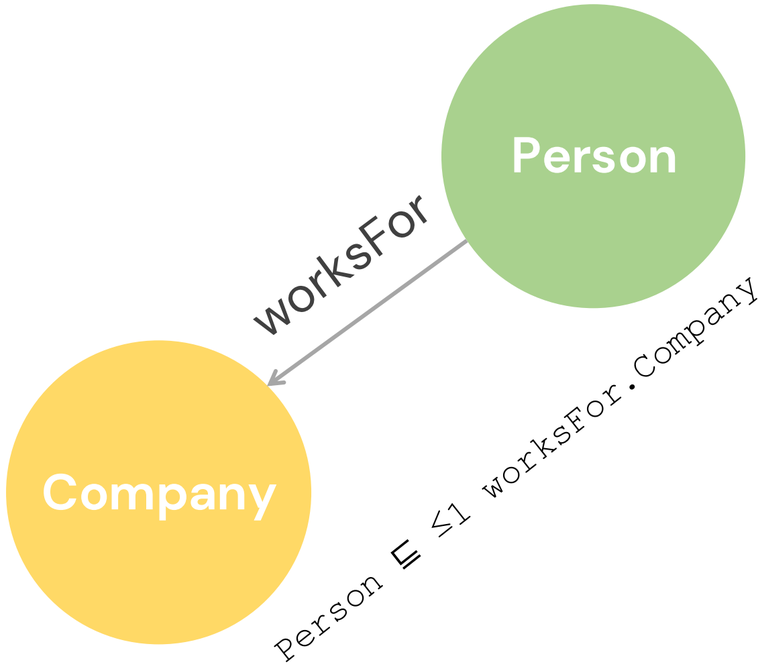
Ontologies

Anatomy of an Ontology

3. Axioms (Constraints & Rules)

Logical statements that define the behavior of the ontology.

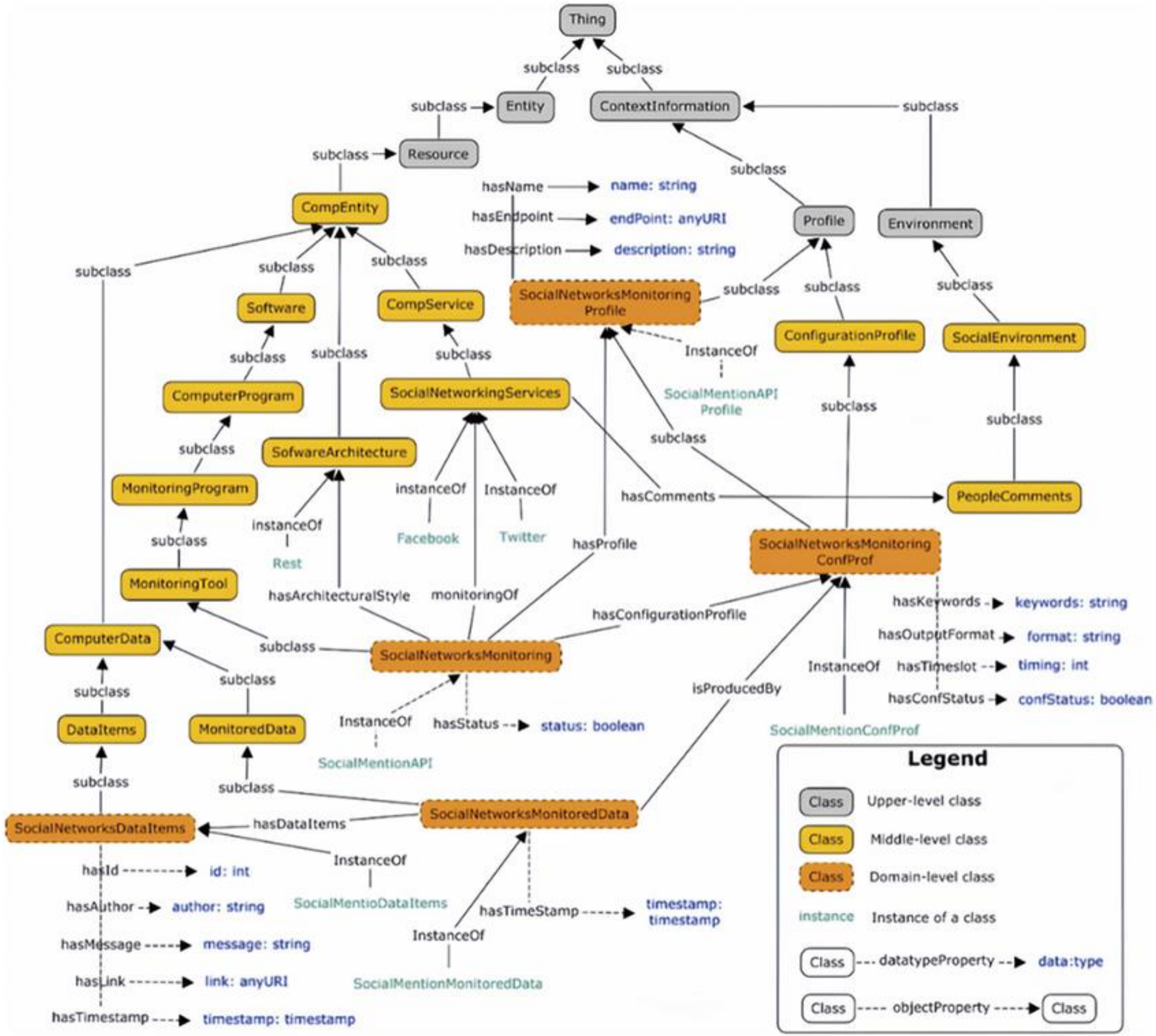
- *Examples:* Person ⊑ hasAge exactly 1 (Every Person has exactly one age).
- *Visual:* **Dashed lines or logical notation** near the relevant classes.



Ontologies

Anatomy of an Ontology

Smart home ontology created by Protégé (OntoGraf Diagram)



Zhao, L., & Lee, S. W. (2025). Intelligent Spatial Anomaly Activity Recognition Method Based on Ontology Matching. *Computers, Materials & Continua*, *83*(3).

Ontologies

➡ Open World vs. Closed World Assumption

CLOSED WORLD (CWA)

Used by: Relational Databases

If it is not stored → it is **FALSE**.

Query: 'Patients with no allergies'
→ Returns all patients not in allergy table.

Safe for applications that control all data.

OPEN WORLD (OWA)

Used by: Ontologies / KGs

If it is not stated → it is **UNKNOWN**.

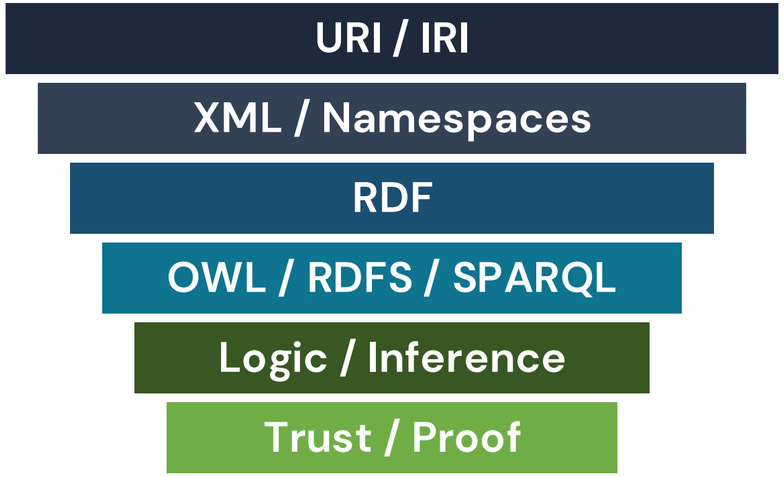
Query: 'Patients with no allergies'
→ Cannot conclude absence!

Realistic for distributed, incomplete data.

The Semantic Web Vision

Tim Berners-Lee, 2001 — 'The Semantic Web'

The Semantic Web is a vision about an extension of the existing World Wide Web, which provides software programs with machine-interpretable **metadata** of the published information and data. In other words, we add further data descriptors to otherwise existing content and data on the Web.

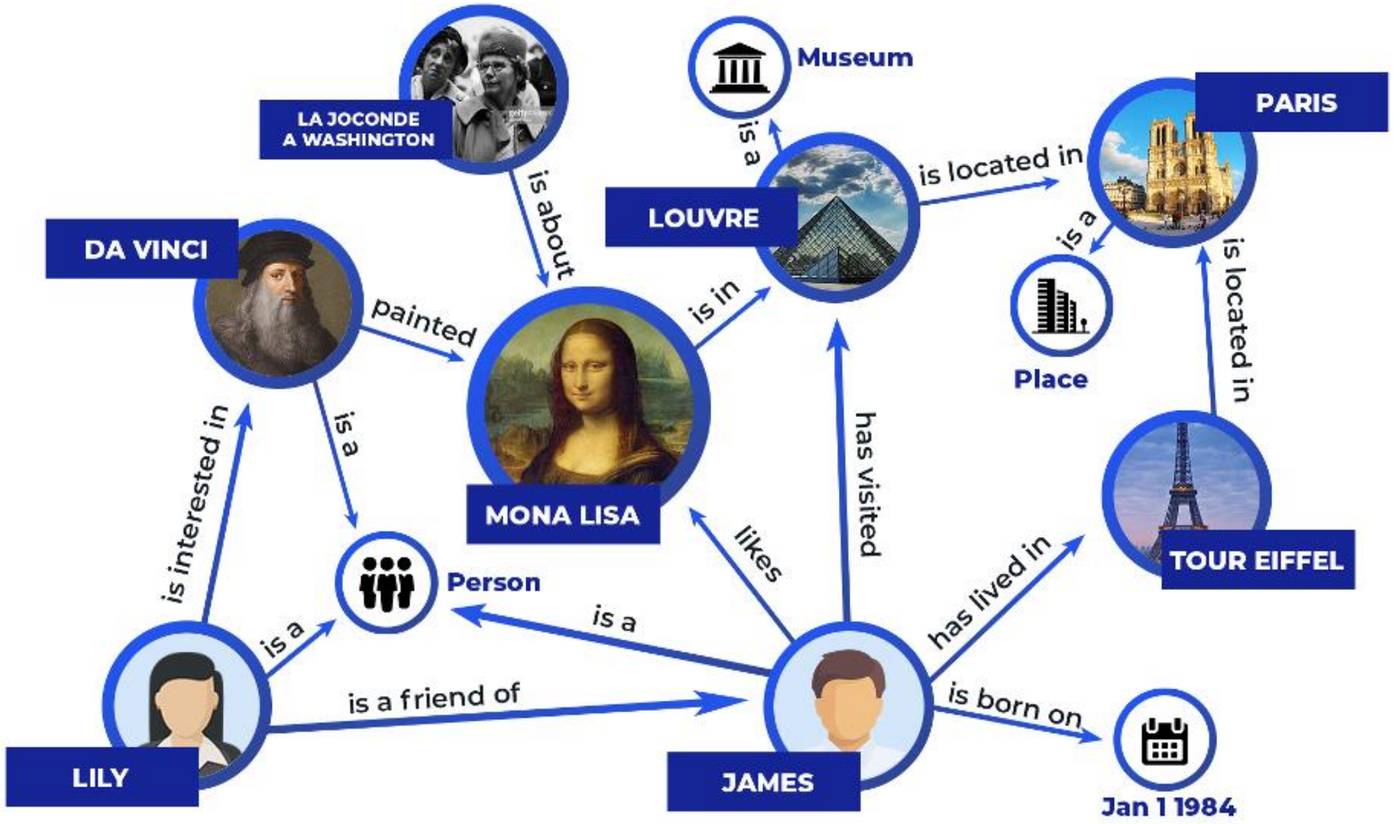


The vision: *any machine can understand any data on the web by shared semantics.*

Knowledge Graphs

➡ What is a Knowledge Graph?

"Suppose I could program my computer to create a space in which anything could be linked to anything"
- Tim Berners-Lee



Knowledge Graphs

What is a Knowledge Graph?

A Knowledge Graph (KG) is a **data structure** that uses **graphs** to represent **data** through **entities (nodes)** and **relationships (edges)**, forming an interconnected **semantic** network.

Objective: To represent knowledge in a **flexible**, **interconnected**, and **inferable** way.

- A KG is **dynamic**, i.e., it can provide new inferences and remodel itself with the addition of new data over time, **without manual user intervention** or the database being explicitly pre-structured.
- Another word for **graphs** is "**networks**".

Knowledge Graphs

What is a Knowledge Graph?

Architecture: The basic element of a KG is a **triple**: <subject, predicate, object>.

ENTITIES

Nodes represent real-world objects: people, places, concepts, and products.

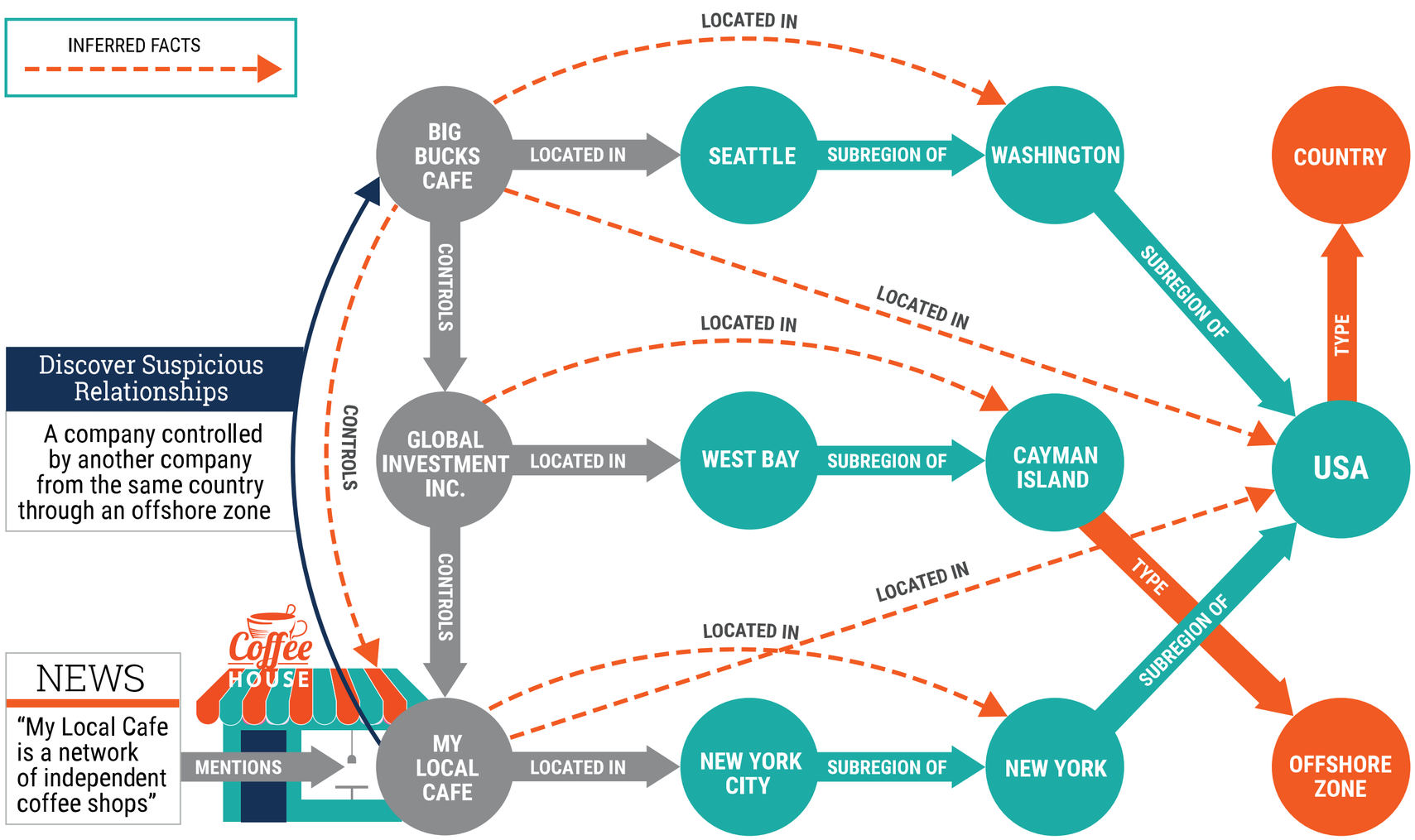
RELATIONSHIPS

Edges represent connections with semantic meaning between entities.

CONTEXT

Properties and metadata enrich nodes and relationships with additional information.

Knowledge Graphs



https://www.ontotext.com/wp-content/uploads/2020/02/What-is-Knowledge-Graph-Offshore-Zone-Case.png

Knowledge Graphs

Knowledge Graph vs. Ontology

Ontology
Defines the meaning. It defines concepts (classes), properties, and rules.

Knowledge Graph
Uses the ontology. It instantiates the concepts with real data.

Result: The ontology functions as a semantic schema for the knowledge graph.

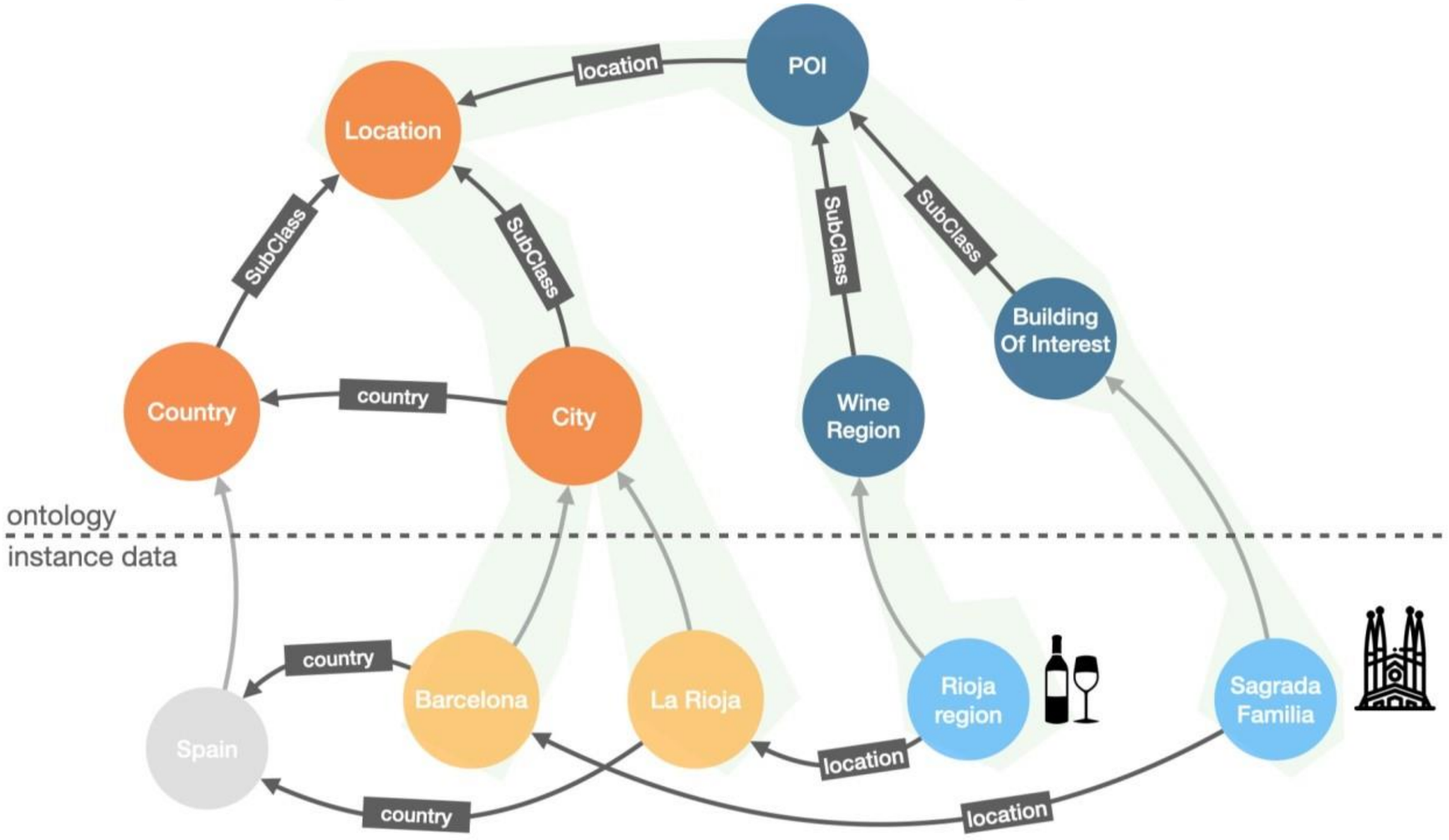
EXAMPLE:

Ontology (model)	Knowledge Graph (data)
Class: Person	John — worksAt → Microsoft
Class: Company	Ana — worksAt → University of Minho
Relationship: worksAt	

- **Ontology** = vocabulary + rules
- **Knowledge graph** = facts connected using this vocabulary

Knowledge Graphs

➡ Knowledge Graph vs. Ontology



Knowledge Graphs

➡ Knowledge Graph vs. Ontology

Aspect	Ontologies	Knowledge Graph
Definition	Formal model that defines concepts, categories, and relationships of a domain	Data structure that represents knowledge as a graph of entities and relationships
Objective	Define the meaning and semantics of a domain	Store and link information in a navigable and searchable way
Nature	Conceptual / semantic model	Database / data structure
Main components	Classes, properties, axioms	Nodes (entities), edges (relationships), properties
Example	Model that defines what Person, Organization, worksAt is	In Graph with: “Ana → worksAt → Company X”
Level of abstraction	More abstract and theoretical	More practical and data-oriented

Knowledge Graphs

Knowledge Graph vs. Ontology

Intuitive way of viewing

Analogy	Ontology	Knowledge Graph
Database	Database schema	Records in the database
Language	Grammar	Sentences
Map	Map legend and rules	The map with roads and cities

Short summary:

- Ontologies define what things mean.
- Knowledge graphs store interconnected facts.
- Typically, knowledge graphs are built using ontologies.

Knowledge Graphs

Why graphs instead of tables?

SQL/TABLES

- **Rigid** – all rows in a table must share the same structure;
- **Expensive JOINs** for deep relationships;
- Querying connected data across many tables quickly becomes **unreadable** SQL;
- No native way to express the ***meaning*** of a relationship.

KNOWLEDGE GRAPHS

- Schema-**flexible**: add a new node or edge type at any time without migrations;
- Relationships are first-class citizens — **no JOINs**:
- **Recursive** and **reachability** queries are **native** and **efficient**;
- **Edge** types carry explicit **semantic meaning**, especially when backed by an ontology

Knowledge Graphs

Property graphs vs. RDF/triples

There are two dominant KG models:

Property Graphs → used by Neo4j

Key characteristics:

- Very flexible
- Easy to query (Cypher)
- Supports rich metadata on both nodes and edges
- Close to how developers think

RDF / Triples → used in semantic web standards

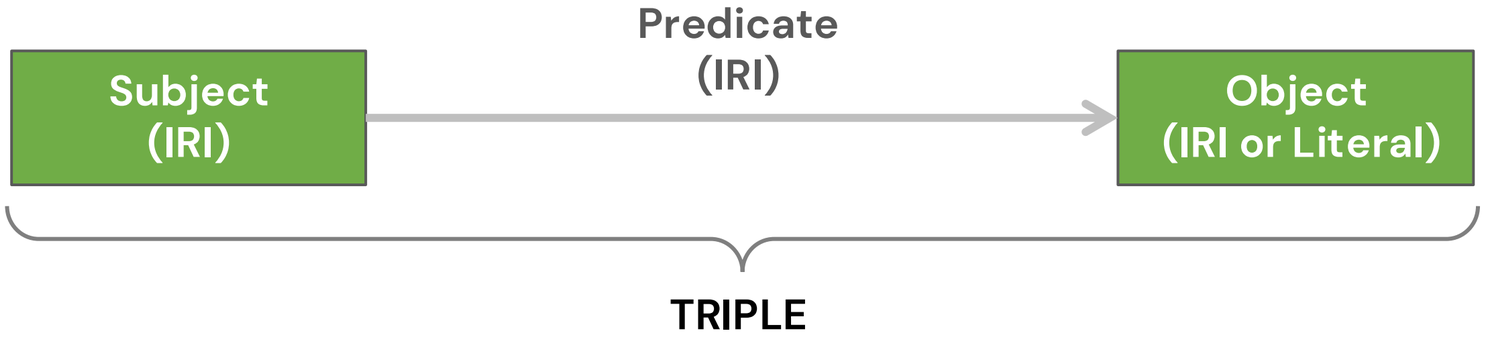
Key characteristics:

- Standardized (interoperability)
- Strong semantics (meaning is explicit)
- Good for **data integration across systems**

RDF and Triples

➡ RDF (Resource Description Framework)

RDF is the **basic data representation model** in the Semantic Web. Everything is represented as **triples**:



The Triple Structure

1. **Subject:** The resource (URI or blank node).
2. **Predicate:** The property or relationship (URI).
3. **Object:** The value (URI or literal).

:Ana rdf:type :Pessoa

:Ana :worksAt :Google

subject predicate object

RDF and Triples

➡ RDF (Resource Description Framework)

:Ana rdf:type :Pessoa

:Ana :worksAt :Google

subject *predicate* *object*

To represent:
Ana works at Google since 2020
You need multiple triples:
:Ana → :worksAt → :Google
:Ana → :worksSince → :2020

⚠ **Important constraint:** No native properties on edges or nodes

A **namespace** is a **shortcut (prefix)** for a long URI.

RDF uses URIs to uniquely identify everything.

- Full URIs are **long** and **ugly**:
  http://xmlns.com/foaf/0.1/name
- With a namespace, you write:
  foaf:name

RDF and Triples

➡ RDF: The Data Model

IRI (URI) Nodes

- Represent real-world entities or concepts
- Also called resources
- Identified by a unique global identifier (URI/IRI)

e.g.:
http://example.org/Person/Ana

Literal Nodes

- Represent values (data)
- Strings, numbers, dates, etc.
- Cannot be subjects or predicates

e.g.:
"30"^^xsd:integer
"Ana"@pt

Blank Nodes (Anonymous Nodes)

- Represent unknown or unnamed entities
- No global identifier (no URI)
- Used when:
  You don't know the identity
  You don't care about naming it

RDF is very strict:
•**Predicates must always be URIs**
Literals = **data values**
URIs = **things/concepts**
Blank nodes = **"something exists, but unnamed"**

RDF and Triples

RDF Schema (RDFS)

It allows the definition of hierarchies (classes/subclasses), property constraints (domain/range), and relationships between resources, acting as a lightweight ontology language to make RDF data machine-understandable.

rdfs:Class
Declares a class

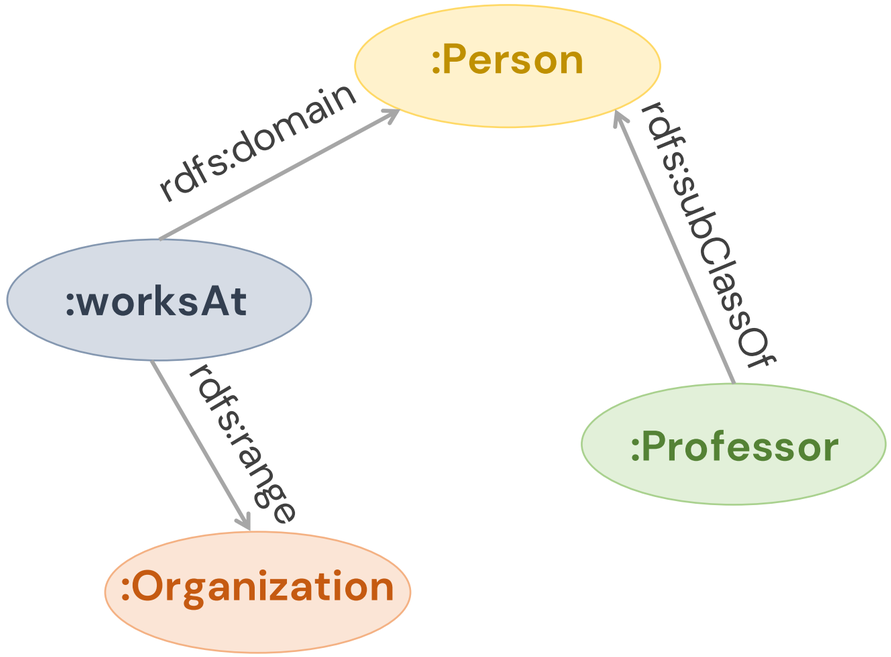
rdfs:subClassOf
Class hierarchy (inheritance)

rdfs:subPropertyOf
Property specialization

rdfs:domain
Subject type of a property

rdfs:range
Object type of a property

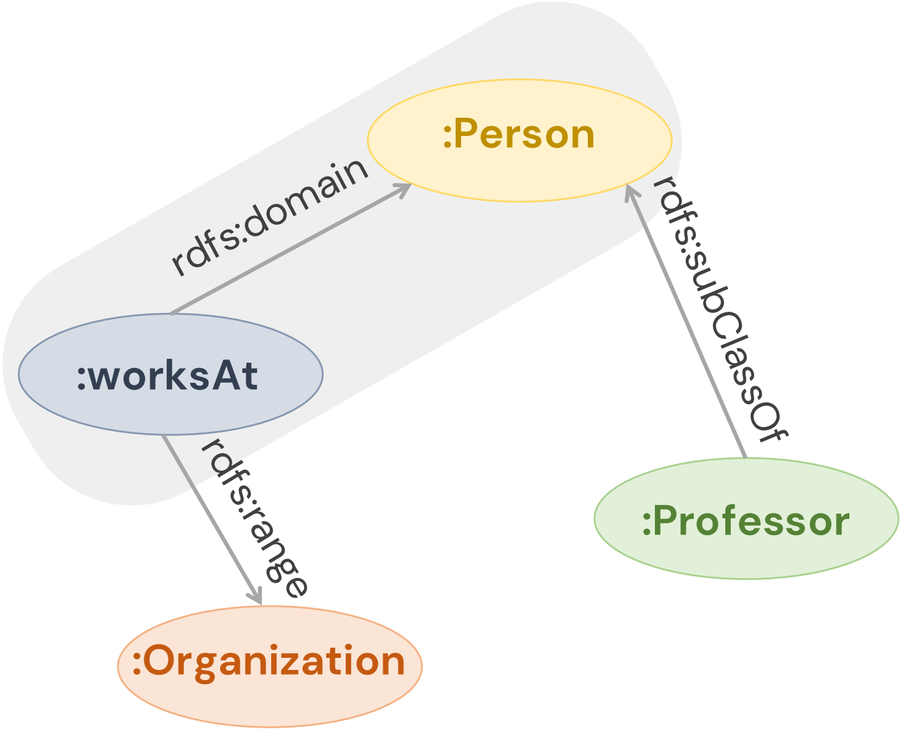
rdfs:label / comment
Human-readable documentation



⚠ RDF provides the **graph model** for facts; RDFS provides the **vocabulary** for describing the meaning of these graphs and the relationships between object types.

RDF and Triples

➡ RDF Schema (RDFS)



rdfs:domain defines which class a property is subject to.

Formally: P rdfs:domain C

↳ **Meaning:** If something uses the property `P` as its subject, then it belongs to class `C`.

EXAMPLE

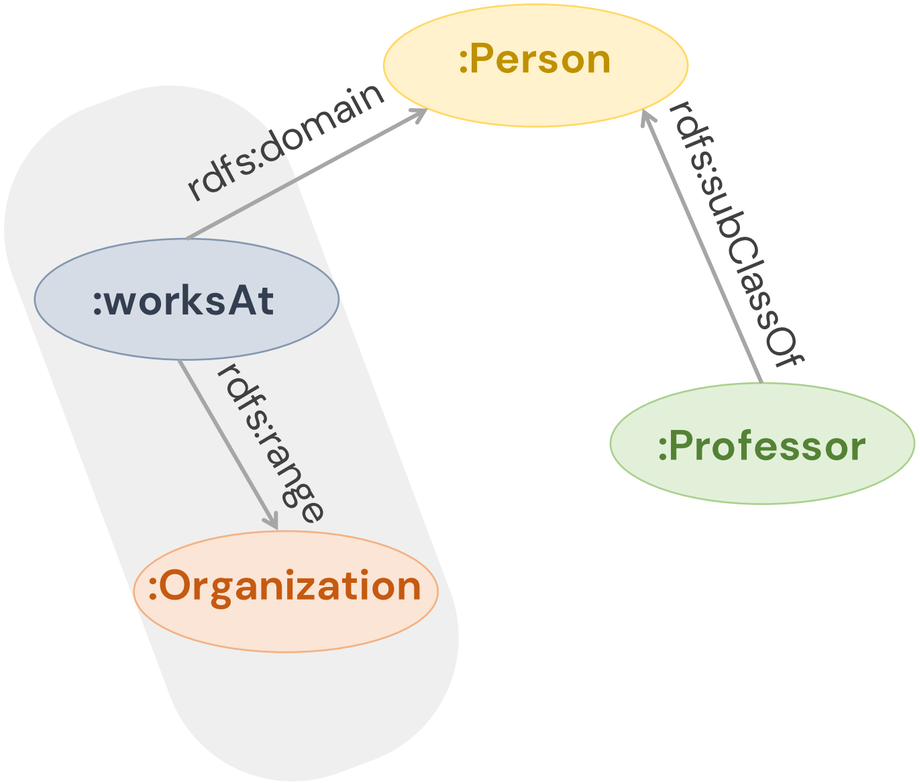
If there is a triple: :João :worksAt :University of Minho

then we can infer: :João rdf:type :Person

That is: whoever uses `worksAt` as its subject is a `Person`.

RDF and Triples

➡ RDF Schema (RDFS)



rdfs:range defines the class of the property object.

Formally: P rdfs:range C

↳ **Meaning:** If something appears as an object of property `P`, then it belongs to class `C`.

EXAMPLE

If there is a triple: :João :worksAt :University of Minho

then we can infer: :University of Minho rdf:type :Organization

That is: whoever is the object of `worksAt` is an `Organization`.

RDF and Triples

➡ RDF Schema (RDFS)

LIMITATIONS: **Cardinality (how many values a property can have)**

In many domains, we want to limit how many values a property can have. For instance, a person has only one date of birth.

:birthDate rdfs:domain :Person
:birthDate rdfs:range xsd:date

This means:
- the subject is a Person
- the object is a date

But with RDFS we cannot say:
that a person has exactly 1 birthdate

Therefore, the following would be allowed in RDFS:

:Ana :birthDate "1990-01-01"
:Ana :birthDate "1991-02-02"

RDF and Triples

RDF Schema (RDFS)

LIMITATIONS: **Disjoint classes (classes that cannot share instances)**

With RDFS, we cannot declare two classes to be mutually exclusive. For instance, let's consider:

:Student rdfs:subClassOf :Person
:Professor rdfs:subClassOf :Person

This means:

- the Student is a subclass of Person
- the Professor is a subclass of Person

But with RDFS we cannot say:
That a person can not be both Student and Professor simultaneously.

Therefore, the following would be allowed in RDFS:

:Diana Ferreira rdf:type :Student
:Diana Ferreira rdf:type :Professor

RDF and Triples

➡ OWL

RDFS is a simple ontology language.

↓

For more **complex** logical constraints, use **OWL** (Web Ontology Language), which is built on RDFS.

⊓ intersectionOf Person ⊓ Employee
All individuals in BOTH classes

⊔ unionOf Cat ⊔ Dog
Individuals in EITHER class

¬ complementOf ¬ Animal
Individuals NOT in the class

∃ someValuesFrom ∃ hasChild.Person
Existential restriction

∀ allValuesFrom ∀ eats.PlantFood
Universal restriction

≥ / ≤ cardinality ≥2 hasAuthor.Person
Min/max cardinality bounds

RDF and Triples

➡ Turtle (Terse RDF Triple Language)

- Turtle is one of the **easiest** and most **readable** RDF formats.
- It is a compact, human-readable syntax for writing RDF.
- Avoids repetition and makes triples cleaner than raw RDF/XML.

RDF FORMATS

Turtle	Readable, compact, prefixes
N-Triples	Simple, stream processing
JSON-LD	Web/APIs, embedded in HTML
RDF/XML	Legacy, W3C original

Prefixes (Namespaces)
Turtle uses @prefix to define namespaces:
@prefix foaf: <http://xmlns.com/foaf/0.1/> .
@prefix ex: <http://example.org/> .

Basic Triple
ex:Ana foaf:name "Ana" .
Means:
- Subject: Ana
- Predicate: name
- Object: "Ana"

RDF Types (Classes)
To say something “is a” type:
ex:Ana a foaf:Person .
a is shorthand for:
rdf:type

RDF and Triples

➡ Turtle (Terse RDF Triple Language)

Multiple Properties (Same Subject)

Instead of repeating the subject:

```
ex:Ana foaf:name "Ana" .
ex:Ana foaf:age 25 .
```

Turtle allows:

```
ex:Ana
   foaf:name "Ana" ;
   foaf:age 25 .
```

- ; → same subject, new predicate
- . → end of statement

Multiple Objects (Same Predicate)

```
ex:Ana foaf:knows ex:Bob .
ex:Ana foaf:knows ex:Maria .
```

Turtle version:

```
ex:Ana foaf:knows ex:Bob , ex:Maria .
```

- , → same subject + predicate

RDF and Triples

➡ Turtle (Terse RDF Triple Language)

Turtle (.ttl)

```
@prefix foaf: <http://xmlns.com/foaf/0.1/> .
@prefix ex:   <http://example.org/> .
@prefix xsd:  <http://www.w3.org/2001/XMLSchema#> .

ex:Alice
  a foaf:Person ;
  foaf:name "Alice"@pt ;
  foaf:age "30"^^xsd:integer ;
  foaf:knows ex:Bob .

ex:Bob
  a foaf:Person ;
  foaf:name "Bob"@pt ;
  ex:trabalhaEm ex:Neo4j .

ex:Neo4j
  a ex:Empresa ;
  foaf:name "Neo4j Inc." ;
  ex:fundadoEm "2007"^^xsd:gYear .
```

RDF and Triples

➡ SPARQL

- SPARQL = query language for RDF graphs
- Similar to SQL, but navigates triple patterns.

BASIC QUERY STRUCTURE

```
SELECT ?x
WHERE {
  ?x ?y ?z .
}
```

- SELECT → what you want back
- WHERE → pattern to match
- ?x ?y ?z → variables - start with ? represent unknown values

Example:

Turtle data:

ex:Ana foaf:name "Ana" .

Query:

```
SELECT ?name
WHERE {
  ex:Ana foaf:name ?name .
}
```

Result:

"Ana"

RDF and Triples

➡ SPARQL

Multiple Patterns (Graph Matching) ⟶ Works like a JOIN in SQL

```
SELECT ?name
WHERE {
ex:Ana foaf:knows ?person .
?person foaf:name ?name .
}
```

FILTER (Conditions) ⟶ Works like WHERE in SQL

```
SELECT ?titulo ?ano
WHERE {
  ?filme a ex:Filme .
  ?filme ex:titulo ?titulo .
  ?filme ex:ano ?ano .
  FILTER (?ano > 2000)
  FILTER (lang(?titulo) = "pt")
}
ORDER BY DESC(?ano)
```

RDF and Triples

➡ SPARQL

SELECT DISTINCT ⟶ Removes duplicates

```
SELECT DISTINCT ?person
WHERE {
 ?person foaf:knows ?x .
}
```

LIMIT ⟶ Limits the number of results

```
SELECT ?person
WHERE {
  ?person foaf:name ?name .
}
LIMIT 5
```

OPTIONAL

```
SELECT ?name ?age
WHERE {
 ?p foaf:name ?name .
 OPTIONAL { ?p foaf:age ?age }
}
```

- If age exists → return it
- If not → still return name

RDF and Triples

➡ SPARQL

ASK ⟶ ASK checks whether a pattern exists in the graph and returns only **true or false**

Turtle data:
ex:Ana foaf:knows ex:Bob .

Query:
```
ASK {
ex:Ana foaf:knows ex:Maria .
}
```

Result:
```
false
```

"Does Ana know *anyone*?"

```
ASK {
ex:Ana foaf:knows ?person .
}
```

"Is there anyone older than 30?"

```
ASK {
?p foaf:age ?age .
FILTER (?age > 30)
}
```

Property Graphs

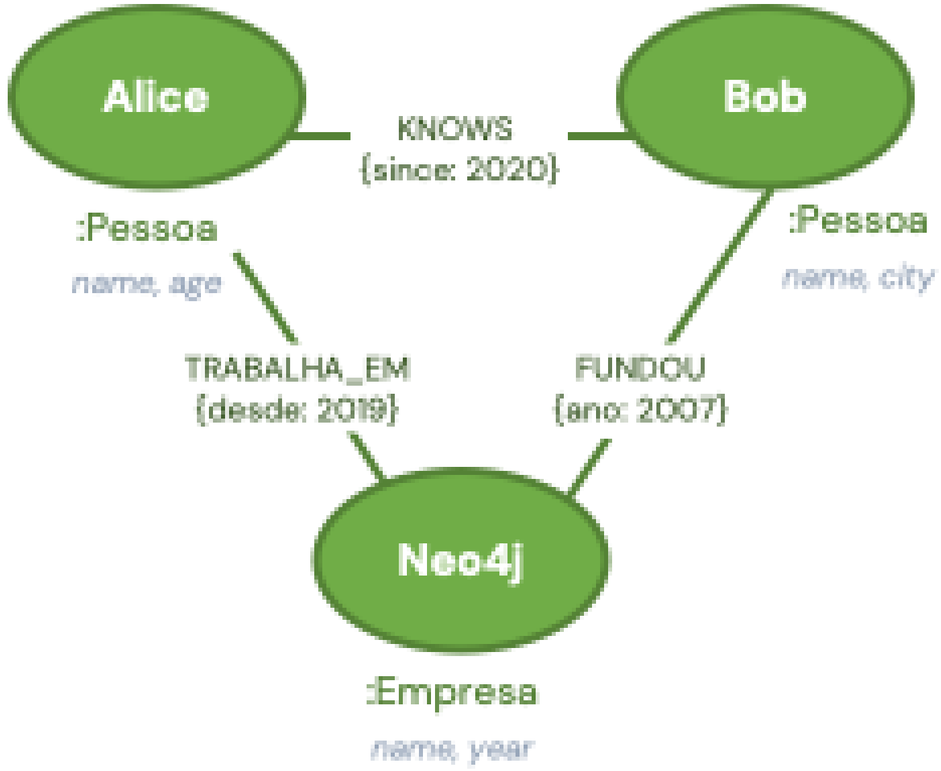
➡ Neo4j

- **Nodes** (entities)
Represents an entity. It can have one or more Labels (e.g., :Person, :Product).

- **Relationships** (edges)
A directed edge between two nodes. It has a mandatory type (e.g., KNOWS, BOUGHT).

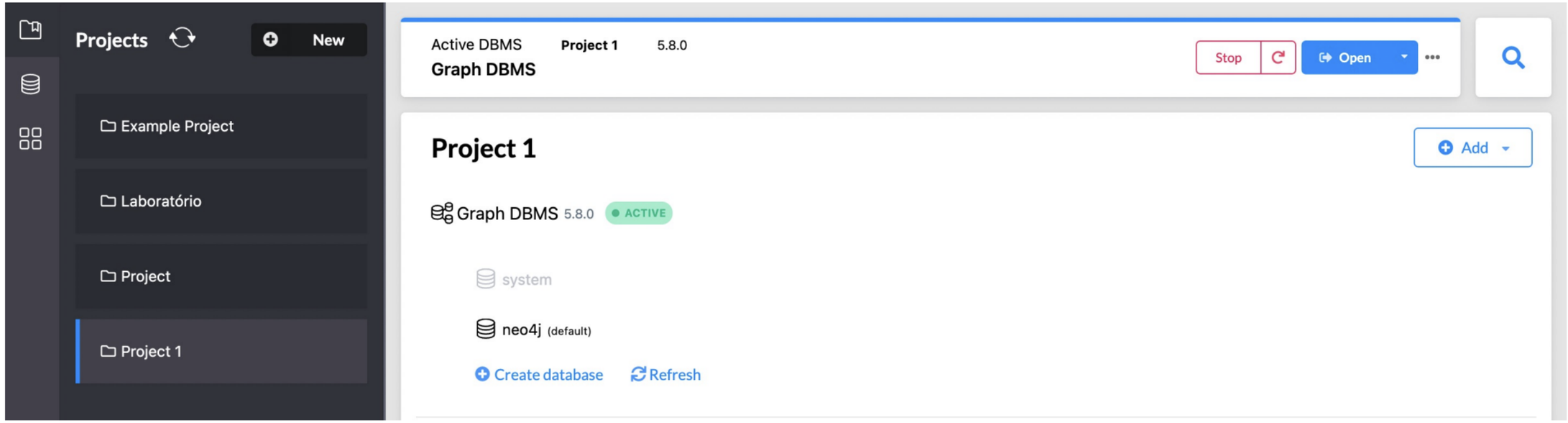
- **Properties** (key-value pairs)
Key-value pair in nodes or relationships. Typed: String, Int, Float, Boolean, Date...

- **Labels** (types of nodes)
It categorizes nodes. A node can have multiple labels. Used for efficient indexing.



Property Graphs

➡ Neo4j Desktop



https://neo4j.com/download/

Property Graphs

➡ Cypher

- Cypher is Neo4j's declarative graph query language.
- It was created in 2011 by Neo4j engineers as an SQL-equivalent language for graph databases.

It relies on the following ascii-art type of syntax:

(nodes)-[:CONNECT_TO]→(otherNodes)

Case sensitive:
• Node labels
• Relationship types
• Property keys

Case insensitive:
• Cypher keywors (return, match, where, etc)

Property Graphs

➡ Cypher: AsciiArt for the nodes

- Nodes are represented by parentheses:
  () or (p)
- Labels, or tags, start with : and group nodes by roles or types: **(p:Person:Mammal)**
- Nodes have properties:
  (p:Person {name : 'Veronica'})

()
(matrix)
(:Movie)
(matrix:Movie)
(matrix:Movie {title: "The Matrix"})
(matrix:Movie {title: "The Matrix", released: 1997})

() represents an uncharacterized, anonymous node. To be used elsewhere in the query, a variable must be added, for instance: (matrix). The variable is only usable in one run.

By using the "Movie" label in conjunction with the : the type of the node is declared. Thus, it is possible to filter the nodes that will be selected by the query.

Property Graphs

➡ Cypher: AsciiArt for the relations

Relationships are represented by hyphens or square brackets:
--> or - [h:HIRED] ->

The direction of the relationships is represented by the < or > sign:
(p1) - [:HIRED] -> (p2) or (p1) <- [:HIRED] - (p2)

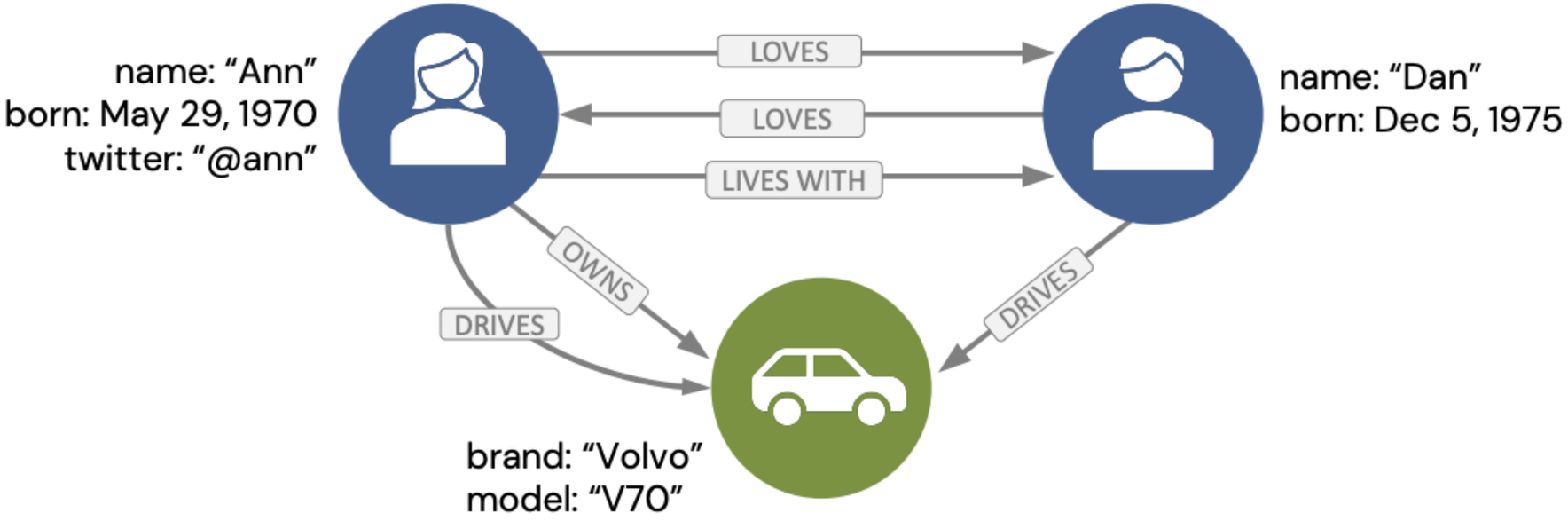
Relationships also have properties:
- [:HIRED {type: 'full-time'}] ->

-->
-[role]->
-[:ACTED_IN]->
-[role:ACTED_IN]->
-[role:ACTED_IN {roles: ["Neo"]}]->

Cypher uses a pair of hyphens to represent an undirected relationship. Directed relationships are represented by the sign of > or <. The square brackets are used to define the details of the relationship.

Property Graphs

➡ Cypher: Example



Who drives a car owned by a lover?

```
MATCH
(p1:Person)-[:DRIVES]->(c:Car)-[:OWNS]->(p2:Person)<-[:LOVES]-(p1)
RETURN
p1
```

- MATCH and RETURN are Cypher keywords
- p1, p2 , c are variables
- :Person, :Car are node labels
- :DRIVES, :OWNS, :LOVES are relationships

Property Graphs

➡ Cypher: Keywords

MATCH: Used to specify patterns to match in the graph.

CREATE: Used to create nodes, relationships, or paths in the graph.

RETURN: Specifies what data to retrieve from the graph.

WHERE: Allows filtering of results based on specified conditions.

MERGE: Combines the functionality of MATCH and CREATE, ensuring that a pattern exists and creating it if not.

DELETE: Removes nodes, relationships, or paths from the graph.

SET: Updates properties of nodes or relationships.

WITH: Chains queries together, passing results from one part of the query to the next.

UNWIND: Expands a list or collection into a sequence of rows.

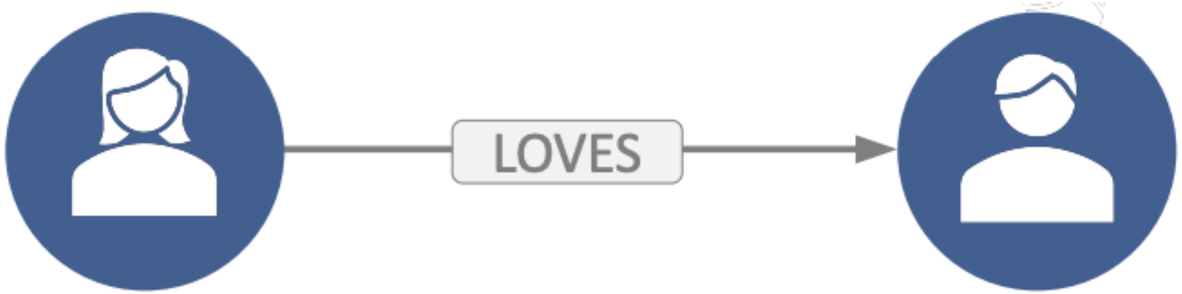
FOREACH: Executes a subquery for each element in a list or collection.

ORDER BY: Sorts the results based on specified criteria.

LIMIT: Limits the number of results returned by the query.

Property Graphs

➡ Cypher: Create data



NODE NODE

```
CREATE (:Person {name: “Ann” }) - [:LOVES] -> (:Person {name: “Dan” })
```

LABEL PROPERTY LABEL PROPERTY

Property Graphs

Cypher: Create data – Singularity

{name: “Ann” }) {name: “Ann” })

```
CREATE (:Person {name: “Ann” })
```

ERROR Neo.ClientError.Schema.ConstraintValidationFailed

```
Neo.ClientError.Schema.ConstraintValidationFailed:
Node(0) already exists with label `Person` and
property `name` = 'Ann'
```

```
CREATE CONSTRAINT ON (p:Person)
ASSERT p.name is UNIQUE
```

Property Graphs

➡ Cypher: Create data and search nodes

```
CREATE (me:Person {name: "My Name" })
RETURN me
```

```
MATCH (me:Person )
WHERE me.name="My Name"
RETURN me.name
```

Short-hand syntax:

```
MATCH (me:Person {name:"My Name"})
RETURN me.name
```

Property Graphs

➡ Cypher

OPERATORS:

```
=, <>, <, >, <=, >=, IS NULL, IS NOT NULL
```

REGULAR EXPRESSIONS:

```
MATCH (p:Person)-[r:ACTED_IN]->(m:Movie)
WHERE p.name =~ "K.+" OR m.released >= 2000
RETURN p, r, m
```

FILTER RESULTS:

```
MATCH (m:Movie {title: "The Matrix"})
RETURN m

MATCH (m:Movie)
WHERE m.title = "The Matrix"
RETURN m
```

RUN QUERIES:

```
:play movie-graph
```

```
MATCH (p:Person {name: "Tom Hanks"})-[r:ACTED_IN|DIRECTED]-(m:Movie)
RETURN p,r,m

MATCH (p:Person {name: "Tom Hanks"})-[r:ACTED_IN|DIRECTED]-(m:Movie)
RETURN p.name, type(r), m.title
```

Property Graphs

➡ Cypher

ORDER BY:

```
MATCH (person:Person)
RETURN person.name, person.born
ORDER BY person.born
```

SKIP AND LIMIT:

```
MATCH (actor:Person) -[:ACTED_IN]->(movie:Movie)
RETURN actor.name AS Actor, movie.title AS Movie
SKIP 10 LIMIT 10
```

ADD NODES:

```
CREATE (movie:Movie {title: "Mystic River", released:1993 })
```

ADD/UPDATE PROPERTIES:

```
MATCH (movie:Movie)
WHERE movie.title = "Mystic River"
SET movie.tagline = "We bury our sins here, Dave. We wash them clean."
RETURN movie.title AS title, movie.tagline AS tagline
```

Property Graphs

➡ Cypher

CREATE RELATIONSHIPS:

Find the movie "Mystic River" and actor Kevin Bacon and create a relationship between them.

```
MATCH (kevin:Person) WHERE kevin.name = "Kevin Bacon"
MATCH (mystic:Movie) WHERE mystic.title = "Mystic River"
CREATE (kevin)-[r:ACTED_IN {roles:["Sean"]}]->(mystic)
RETURN mystic, r, kevin
```

OR

```
MATCH (p:Person {name: "Kevin Bacon"})
MATCH (m:Movie {title: "Mystic River"})
MERGE (p)-[r:ACTED_IN]->(m)
ON CREATE SET r.roles = ["Sean"]
RETURN p, r, m
```

MERGE = MATCH + CREATE

Knowledge Graphs

➡ Property graphs vs. RDF/triples

Feature	Property Graph	RDF (Resource Description Framework)
Standard	No universal standard	W3C (Semantic Web)
Structure	Nodes + Edges + Properties	Triple (Subject-Predicate-Object)
Flexibility	Very high	More rigid
Properties	On nodes AND edges	Only via extra triples
Language	Cypher (Neo4j)	SPARQL
Focus	Performance, Transactions (OLTP)	Interoperability, Inference
Ease of use	Easier	More complex

💡 Both represent knowledge, but optimize for different goals:

- Property Graphs → **performance, usability, applications**
- RDF → **interoperability, standards, data sharing**

Knowledge Graphs

➡ Applications

🔎 **1. Semantic Search (Search Engines)**

Search engines like Google use knowledge graphs to:

- Understand user intent
- Connect related concepts
- Display knowledge panels

Example: Searching for “Cristiano Ronaldo” shows his club, age, statistics, etc.

🛒 **2. Recommendation systems**

Companies like Amazon and Netflix use graphs to:

- Recommend products/movies
- Analyze relationships between users, items, and preferences

Example: “Customers who bought this also bought…”

Knowledge Graphs

➡ Applications

👥 3. Social Networks

Platforms like LinkedIn and Facebook use graphs for:

- Suggestions of friends/connections
- Community detection
- Influence analysis

🤖 4. Chatbots and intelligent assistants

Assistants like Siri or Google Assistant use knowledge graphs for:

- Answering questions with context
- Linking information from multiple sources

Knowledge Graphs

➡ Applications

🧬 **5. Health and Bioinformatics**

- Connecting diseases, genes, and medications
- Drug discovery
- Assisted diagnosis

Example: relationships between symptoms and diseases

🏢 **6. Enterprise Knowledge Management**

- Centralize internal information (*e.g.*, multiple data sources and departments)
- Link documents, people, and processes
- Improve onboarding and productivity

Knowledge Graphs

➡ Applications

📊 **7. Data Integration**

Solving the problem of scattered data:

- Different databases → unified model
- Identify and integrate disparate resources in the Web (*e.g.*, transport, cinemas, films)
- Facilitates analysis and BI

🚗 **8. Mobility and Logistics**

- Route planning
- Transportation networks
- Delivery optimization

Knowledge Graphs

Summary

Knowledge Graphs

They represent entities and relationships with rich semantics. They are fundamental to modern AI, NLP, and data integration.

Property Graphs

Optimized for performance and ease of use. Cypher is intuitive. Ideal for operational applications.

RDF & Triplos

W3C standard for linked data. They enable formal reasoning via OWL. Foundation of the Semantic Web and open data.

Informed Decision

There is no single answer. Performance + simplicity → Property Graph. Semantics + interoperability → RDF.

Kahoot

Goal

- To **reward and compensate students** who actively attend and participate in classes
- To reinforce key concepts in an engaging and competitive way

Rules

- Only students **physically present in class** may participate
- No external help (notes, phones, AI tools, or collaboration)
- Any form of cheating results in **disqualification**

Scoring & Rewards

🥇 **1st place**: +0.5 bonus points

🥈 **2nd place**: +0.3 bonus points

🥉 **3rd place**: +0.2 bonus points

(Bonus points are added to the continuous assessment component.)

⚠️ Bonus points are **non-transferable** and **cannot exceed course limits**

Practical Application

➡ **Worksheet PL7**

Blackboard